
信息安全综合实验：简单入侵检测模型的设计与实现

1.1 实验目的与要求

入侵检测系统(Intrusion Detection System, 简称 IDS)是一种对网络传输进行实时监视,并在发现可疑传输时发出警报或者采取主动防御措施的网络安全设备。作为 IDS 的核心技术,入侵检测通过收集和分析网络行为、安全日志、审计数据以及计算机系统中若干关键点的信息来检查网络与系统中是否存在违反安全策略的行为和被攻击的迹象。根据不同的实现方法,入侵检测分为特征检测与异常检测。特征检测假设入侵行为之间存在相似的特征。它通过对已有入侵行为进行收集和分析,提取出共同的特征,然后利用这些特征预测新的入侵行为。

本实验编程训练的目的如下:

- ① 深入理解特征检测的基本思想, 以及其在入侵检测系统中的应用。
- ② 掌握 K-Means 算法的计算过程。
- ③ 掌握数据挖掘的基本概念与方法。

本实验编程训练的要求如下:

- ① 使用 K-Means 算法对 KDD Cup 1999 数据集进行聚类分析, 建立简单的入侵检测模型。
- ② 利用入侵检测模型对测试数据进行预测。

1.2 相关背景知识

1. KDD Cup 1999 数据集

KDD Cup 1999 数据集是第 3 届知识发现与数据挖掘竞赛所使用的数据集。该数据集来源于一个模拟的美国空军军事网络环境下的局域网流量数据,包含了多种网络攻击与入侵行为。KDD Cup 1999 通过记录上述局域网连续 9 周的原始 TCP dump 数据流量,生成大约 700 万条连接记录。每一条连接记录反映了某一时刻从源 IP 地址到目的 IP 地址若干数据包的传输情况。整个数据集分为训练数据集和测试数据集 2 部分。前者用于提取数据特征,生成数据挖掘模型,后者用于验证模型的效率与正确性。

KDD Cup 1999 对每一条连接记录都进行了分类(label)。所有记录分为正常(normal)与攻击(attack)两大类。其中攻击(attack)行为又分为以下 4 类:

- (1) DOS: 拒绝服务, 例如, SYN 洪泛攻击。
- (2) R2L: 来自于远程主机的非法入侵, 例如, 猜测密码。

(3) U2R: 非法获得本地主机的超级用户权限, 例如, 各种“缓冲区溢出”攻击。

(4) Probing: 监视与探测, 例如, 端口扫描。

除类别 (label) 以外, KDD Cup 1999 数据集中的每一条连接记录还包含了 41 项属性。本实验共用到其中 18 项属性, 如表 10-1 所示。其中前 3 个属性为符号类型, 其它属性为连续类型。

表 10-1 KDD Cup 1999 数据记录的 18 项属性

名称	类型	说明
protocol_type	symbolic	协议类型
Service	symbolic	服务类型
Flag	symbolic	状态标志
src_bytes	continuous	源到目的字节数
dst_bytes	continuous	目的到源字节数
num_failed_logins	continuous	登录失败次数
num_root	continuous	root 用户权限存取次数
Count	continuous	2 秒内连接相同主机数目
srv_count	continuous	2 秒内连接相同端口数目
serror_rate	continuous	"REJ"错误的连接数比率
same_srv_rate	continuous	连接到相同端口数比率
diff_srv_rate	continuous	连接到不同端口数比率
dst_host_srv_count	continuous	相同目的地相同端口连接数
dst_host_same_srv_rat	continuous	相同目的地相同端口连接数比率
dst_host_diff_srv_rate	continuous	相同目的地不同端口连接数比率
dst_host_same_src_port_rate	continuous	相同目的地相同源端口连接比率
dst_host_srv_diff_host_rate	continuous	不同主机连接相同端口比率
dst_host_srv_serror_rate	continuous	连接当前主机有 S0 错误的比率

读者可以登陆 <http://archive.ics.uci.edu/ml/databases/kddcup99/kddcup99.html> 下载 KDD Cup 1999 数据集。值得注意的是, KDD Cup 1999 作为一个完备的数据集, 包含了多个文件。本实验编程将使用文件 kddcup.data_10_percent.gz 与 corrected.gz 分别作为入侵检测模型的训练数据集与测试数据集。

2. K-Means 算法

1. 聚类方法简介

K-Means 算法是一种聚类分析方法。目前在聚类分析领域, 根据具体应用的不同, 聚类算法分为以下 5 类:

- (1) 划分方法
- (2) 层次方法
- (3) 基于密度的方法

(4) 基于网格的方法

(5) 基于模型的方法

本实验使用的 K-Means 算法就是划分方法中的一种。所谓划分方法，就是在一个由 n 个对象或元组组成的数据库中构建数据的 k 个划分 ($k < n$)，每个划分表示一个聚类。也就是说，本方法将数据划分为 k 个聚类，并且同时满足以下 2 个要求：

(1) 每个聚类至少包含一个对象。

(2) 每个对象必须属于且只属于一个聚类。

一般来说，判断划分结果优劣的基本原则是：同一个聚类中的对象之间尽可能“接近”，不同聚类中的对象之间尽可能“远离”。通常，聚类算法会选择一个划分标准（称为相似度函数）来判定对象之间的相似度，比如距离等。下面首先介绍聚类分析中的数据类型以及相似度的计算方法，然后再讨论 K-Means 算法的设计与实现。

2. 聚类分析中的相似度计算方法

聚类分析中的数据类型包括数值属性、二值属性、符号属性、顺序属性和比例属性。因为 K-Means 算法只适用于聚类均值有意义的情况，所以本实验只介绍数值属性的相似度计算方法。数值属性所描述对象之间的相似度可以通过计算两个数据对象之间距离来确定，比如欧氏距离、Manhattan 距离以及 Minkowski 距离等。最常用的距离计算公式就是欧氏距离，定义如下：

$$d(i, j) = \sqrt{\left(|x_{i1} - x_{j1}|^2 + |x_{i2} - x_{j2}|^2 + \cdots + |x_{ip} - x_{jp}|^2 \right)}$$

其中对象 $i = (x_{i1}, x_{i2}, \cdots, x_{ip})$ ；对象 $j = (x_{j1}, x_{j2}, \cdots, x_{jp})$ 。

另一个常用的距离计算方法就是计算 Manhattan 距离，公式定义如下：

$$d(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \cdots + |x_{ip} - x_{jp}|$$

Minkowski 距离是欧式距离和 Manhattan 距离的一个推广，公式定义如下：

$$d(i, j) = \left(|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + \cdots + |x_{ip} - x_{jp}|^q \right)^{1/q}$$

其中 q 为一个正整数；当 $q=1$ 时，它代表 Manhattan 距离计算公式；而当 $q=2$ 时，它代表欧氏距离计算公式。

若每个变量被赋予一个权值 w ，表示其所代表属性的重要性，那么带权值的欧氏距离计算公式如下：

$$d(i, j) = \sqrt{\left(w_1 |x_{i1} - x_{j1}|^2 + w_2 |x_{i2} - x_{j2}|^2 + \cdots + w_p |x_{ip} - x_{jp}|^2 \right)}$$

同样，Manhattan 距离和 Minkowski 距离也可以引入权值进行计算。

3. K-Means 算法设计

(1) K-Means 算法基本思想

K-Means 算法首先随机选取 k 个对象作为初始聚类中心；然后计算各个数据对象到每个聚类中心的距离，把数据对象划分到离它最近的聚类中心所在的类中；对调整后的新类重新计算聚类中心，如果相邻两次计算的聚类中心没有任何变化，说明数据对象的划分结束。本算法的特点是在每次迭代计算中都要考察各个对象的分类是否正确，如果不正确，就需要重新进行分类。在全部数据调整完毕后，再重新计算聚类中心，进入下一次迭代。如果在一次迭代后所有的数据对象都被正确归类，那么聚类中心将不会改变。这也标志着算法已经达到收敛，可以终止聚类过程。

(2) K-Means 算法流程

K-Means 算法的流程用数学语言描述如下。

- a. 给定大小为 n 的数据集，令聚类次数 $I=1$ ，选取 k 个数据对象作为初始聚类中心 $Z_j(I)$, $j=1,2,3,\dots,k$ 。 k 个聚类用 $Cluter(j)$ 表示，其中 $j=1,2,3,\dots,k$ 。
- b. 计算每个数据对象与各聚类中心的距离 $D(x_i, Z_j(I))$, $i=1,2,3,\dots,n$, $j=1,2,3,\dots,k$ ，如果满足 $D(x_i, Z_m(I)) = \min\{D(x_i, Z_j(I)), j=1,2,3,\dots,k\}$ ，那么 $x_i \in Cluter(m)$ 。
- c. 按照下式重新计算每个聚类的聚类中心。其中 n_j 为 $Cluter(j)$ 中数据对象的数目， $x_i^{(j)}$ 表示 $Cluter(j)$ 中第 i 个对象的值。

$$Z_j(I+1) = \frac{1}{n_j} \sum_{i=1}^{n_j} x_i^{(j)}, (j=1,2,\dots,k)$$

- d. 若 $Z_j(I+1) \neq Z_j(I)$ ，则 $I=I+1$ ，返回第 2 步；否则，算法结束。

3. K-Means 算法的缺陷与扩展

1. K-Means 算法的缺陷

虽然 K-Means 算法复杂度低，易于实现，但是在处理大规模数据时，聚类效果往往欠佳。尤其是在对初始聚类中心数目 k 进行设置时，选取不同的 k 值，聚类结果也大不相同。KDD Cup 1999 数据集除了包含大量的正常数据外，还包含了多种攻击数据。虽然这些攻击数据可以划分为 4 大类，但是在使用 K-Means 算法进行聚类时，仅仅将初始聚类中心数目设置成 5（包括正常数据类型的情况）并不能够获得最佳的聚类效果。因为在 KDD Cup 1999 数据集中，正常数据和攻击数据是多种多样的。以 DOS 攻击为例，它包含了 smurf、neptune、back、teardrop、pod 以及 land 共 6 种攻击方式。不同种类的攻击有着不同的数值特性，即使属于同一类攻击方式，在相似度方面也存在很大差异。有些攻击数据在相似度上与正常数据十分接近，却与同类型的攻击数据相去甚远。因此，在使用 K-Means 算法对 KDD Cup 1999 进行聚类分析时，无论 k 值如何选取，仅仅对数据集进行一次聚类是远远不够的。

2. K-Means 算法扩展

为了获得更加精确的聚类结果，需要利用 K-Means 算法对 KDD Cup 1999 数据集进行多次聚类。

对一次聚类而言，它的结果包含了多个划分，每一个划分表示一个子类（SubCluster）。每一个子类中可能只包含了一种类别的数据，也可能包含了多种类别的数据。对于前一种情况而言，说明该子类已经从其它数据中划分出来，代表了唯一一种类别；而对于后一种情况而言，子类中包含着干扰数据，不能用一种类别来对子类进行标记。在一般情况下，一次聚类很难划分出若干个完全干净（即不含任何干扰数据）的子类。子类中往往不只含有一种类别的数据。因此判断一个子类是否可以代表一种类别，是否需要再进行一次聚类，需要引入聚类精度的概念。

聚类精度（Cluster Precision）表示一个子类中干扰数据占主类别数据的比例。它的计算过程分为以下 5 步。

- (1) 统计子类中不同类别（Label）数据的数目，用 LabelNum[1]、LabelNum [2].....表示。
- (2) 在所有类别中找出数据数目最大的一项，将该值记为 MaxLabelNum，并将对应的类别设置为本子类的主类别（MasterLabel）。
- (3) 统计所有非主类别的数据的数目，记为 OtherLabelNum。
- (4) 计算子类的聚类精度 $\text{Cluster Precision} = \text{OtherLabelNum} / \text{MaxLabelNum}$ 。
- (5) 判断聚类精度是否符合要求（即是否低于某一精度标准，比如 0.1），若不是，则使用 K-Means 算法对该子类继续进行聚类；否则，用主类别标记该子类。

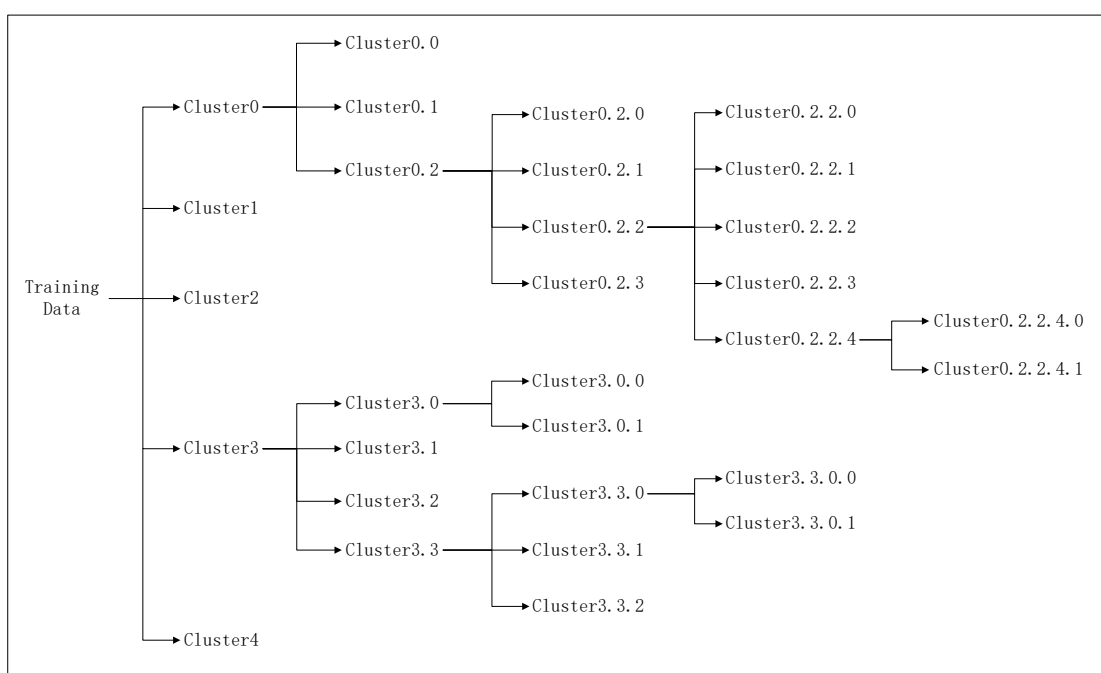


图 1 聚类树

图 1 形象地描述了利用 K-Means 算法进行多次聚类的过程。根节点表示训练数据集。通过 K-Means 算法对其进行一次聚类后，划分出 5 个子类，Cluster0、...、Cluster4。通过计算聚类精度，得出 Cluster1、Cluster2 和 Cluster4 符合精度要求，不再进行任何操作；而 Cluster1 和 Cluster4 不符合要求，需要采用 K-Means 算法继续进行聚类。通过重复上述操作，不符合聚类精度要求的子类不断地被进一步划分，生成了一棵聚类树（Cluster Tree）。树中的每一个叶子节点都代表了一个符合聚类精度要求的子类。聚类树的深度则代表了采用 K-Means 算法进行聚类的次数。

此外，采用 K-Means 算法进行多次聚类是一个自动的过程。对于每次聚类而言，初始聚类中心数目 k 的设定也应该是自动的，不需要用户进行手动输入。因此，本实验规定在对训练数据集进行首次 K-Means 聚类时，将 k 值设置为 5。这样既包含了正常类别，也包含了 4 种攻击类别。在对子类进行 K-Means 聚类时， k 值设为该子类中所包含的不同类别数目。采用这种取值方案，是希望在下一次聚类后将子类划分为不同类别的 k ($k \leq 5$) 个聚类。

1.3 实例编程练习

1. 编程练习要求

本实验要求在 Linux 平台下编写 DataPretreat 与 Kmeans 两个应用程序。DataPretreat 程序负责 KDD Cup 1999 数据集的预处理工作。在完成数据集预处理之后，Kmeans 程序使用 K-Means 算法对训练数据集进行多次聚类分析，生成聚类树，建立入侵检测模型，然后读取测试数据集的数据，预测每条记录的类别。下面分别介绍 DataPretreat 与 Kmeans 两个程序的具体要求。

1. DataPretreat 程序

DataPretreat 是 Linux 命令行程序，命令行格式如下。

`./DataPreTreat [输入文件路径]`

[输入文件路径]指需要进行数据预处理的文件路径。因为无论是对训练数据集进行聚类，还是对测试数据集进行预测，都需要先对数据记录进行预处理，所以 DataPretreat 程序提供了输入不同数据集文件的接口。在进行简单地处理之后，程序将这些数据记录转存到输出文件中。输出文件的命名规则要求在输入文件的名称后面加上“_datatreat”的后缀。如下所示，以 corrected 作为输入文件为例，DataPretreat 程序的执行过程分为 2 个阶段。首先从输入文件中读取每一条记录，并进行数据处理；然后将处理后的记录写入到输出文件中。因为数据文件比较大，所以无论是在读文件还是在写文件的时候，都需要及时地输出日志信息。从上面的执行过程中，可以看出 corrected 文件中总共包含了 311029 条记录。所有经过预处理的数据记录都写入输出文件 corrected_datatreat 中。

```
[root@localhost DataPretreat]# ./DataPreTreat corrected
Start reading records from corrected...
----- 10000    lines have read -----
----- 20000    lines have read -----
```

```

.....
All Records have read ! Total 311029 records !
Start writing records into corrected_datatreat...
----- 10000    lines have written -----
----- 20000    lines have written -----
.....
All Records have written !

```

2. Kmeans 程序

Kmeans 为 Linux 命令行程序，命令行格式如下：

```
./Kmeans
```

与 DataPretreat 程序相比，Kmeans 程序不需要用户输入任何参数。程序默认将 kddcup.data_10_percent_datatreat 与 corrected_datatreat 分别作为训练数据集与测试数据集文件。这两个输入文件都是由对应文件经过 DataPretreat 程序处理后生成的。Log.txt 与 Result.txt 是 Kmeans 程序的两个输出文件。Log.txt 记录了程序在运行过程中的一些日志信息，比如聚类树的生成过程，每次使用 K-Means 算法进行聚类的结果等等。Result.txt 文件用于保存对测试数据集的预测结果。如下所示，Result.txt 文件的内容包括 3 个部分。Classification Result 部分记录了所有测试数据的预测结果。每一行记录包括测试数据的真实类别（True Label）、预测类别（Pre Label）以及在聚类树中所匹配的聚类路径（Cluster Path）。Total Result 部分对所有测试数据进行统计，包括了参加测试的数据总数（Total Test Records），预测正确的数据数目（Right Label Records）以及预测正确率（Right Rate）。Confusion Matrix 部分显示了预测结果的混淆矩阵。在混淆矩阵中，0~4 分别代表了 normal、dos、probe、u2r、r2l 五种不同的类别。矩阵中的 5 列表示预测的分类情况，5 行表示真实的类别。例如，在第 1 行第 2 列的数目是 19，它表示真实类别是 dos 而预测结果是 probe 的测试数据有 19 个。

```

===== Classification Result =====
True Label = normal   Pre Label = normal   Cluster Path = 0.4.4.4.4.0
True Label = normal   Pre Label = normal   Cluster Path = 0.4.4.4.4.0
.....
===== Total Result =====
Total Test Records = 311029   Right Label Records = 299950   Right Rate = 0.96438
===== Confusion Matrix =====
T/P  0      1      2      3      4
0    74775  3596   905    0     46
1    143    223136  19     0     0
2    218    120    2039   0     0
3    31     4       0      0     4
4    5442   535    16     0     0

```

2. 编程训练设计与分析

根据上一小节的要求，需要分别编写 DataPretreat 与 Kmeans 这两个程序。DataPretreat

程序只负责完成简单的数据预处理工作,而 Kmeans 程序不但需要实现核心的 K-Means 算法,还需要提供聚类树的相关操作,实现 K-Means 算法的扩展功能。

1. DataPretreat 程序的设计与实现

DataPretreat 是提供数据预处理功能的辅助程序。它包括 DataPretreat.h 与 DataPretreat.cpp 两个文件。如下所示,头文件 DataPretreat.h 定义了一些基本的数据结构和功能函数。

```
.....
#define MAX_BUF_SIZE 512           //定义字符缓冲区最大值
typedef unsigned char BYTE;        //定义 BYTE 类型

/***** 全局函数 intToString *****/
//将一个 Int 类型的数转换成一个 string
*****/
string intToString(int i)
{
    stringstream s;
    s << i;
    return s.str();
}

/***** 数据记录的结构体定义 *****/
struct strMyRecord
{
    BYTE iProtocolType;    //02 协议类型: 符号变量, 3 类
    BYTE iService;         //03 服务类型: 符号变量, 66 类
    BYTE iStatusFlag;      //04 状态标志: 符号变量, 11 类
    int iSrcBytes;          //05 源到目的字节数: 连续变量
    int iDestBytes;        //06 目的到源字节数: 连续变量
    int iFailedLogins;      //11 登录失败次数: 连续变量
    int iNumofRoot;        //16 root 用户权限存取次数: 连续变量
    int iCount;            //23 2 秒内连接相同主机数目: 连续变量
    int iSrvCount;         //24 2 秒内连接相同端口数目: 连续变量
    BYTE iErrorRate;       //27 "REJ"错误的连接数比率: 连续变量: 0 ~ 100
    BYTE iSameSrvRate;     //29 连接到相同端口数比率: 连续变量: 0 ~ 100
    BYTE iDiffSrvRate;     //30 连接到不同端口数比率: 连续变量: 0 ~ 100
    int iDstHostSrvCount;  //33 相同目的地相同端口连接数: 连续变量:
    //34 相同目的地相同端口连接数比率: 连续变量: 0 ~ 100
    BYTE iDstHostSameSrvRate;
    //35 相同目的地不同端口连接数比率: 连续变量: 0 ~ 100
    BYTE iDstHostDiffSrvRate;
    //36 相同目的地相同源端口连接比率: 连续变量: 0 ~ 100
    BYTE iDstHostSameSrcPortRate;
```



```

//37 不同主机连接相同端口比率：连续变量：0~100
BYTE iDstHostSrvDiffHostRate;
//39 连接当前主机有 S0 错误的比率：连续变量：0~100
BYTE iDstHostSrvErrorRate;
BYTE iLabel;                //42 类型标签：        符号变量 5 类
};

```

结构体 `strMyRecord` 用于保存从输入文件中读取的数据记录。在 KDD Cup 1999 数据集中，除类别标签（label）以外，每一条记录由 41 项不同的属性值组成。根据 K-Means 算法的要求，结构体 `strMyRecord` 没有包含所有属性，而是选取了其中的 18 项属性。这些属性分为符号变量与连续变量 2 种类型。符号变量包括协议类型、服务类型、状态标志以及最后的类别标签。因为 K-Means 算法只适用于均值有意义的情况，所以需要将符号变量用数值表示。例如，对协议类型属性而言，就可以用数值 0、1、2 分别代替 `icmp`、`tcp` 和 `udp` 三种不同的协议。

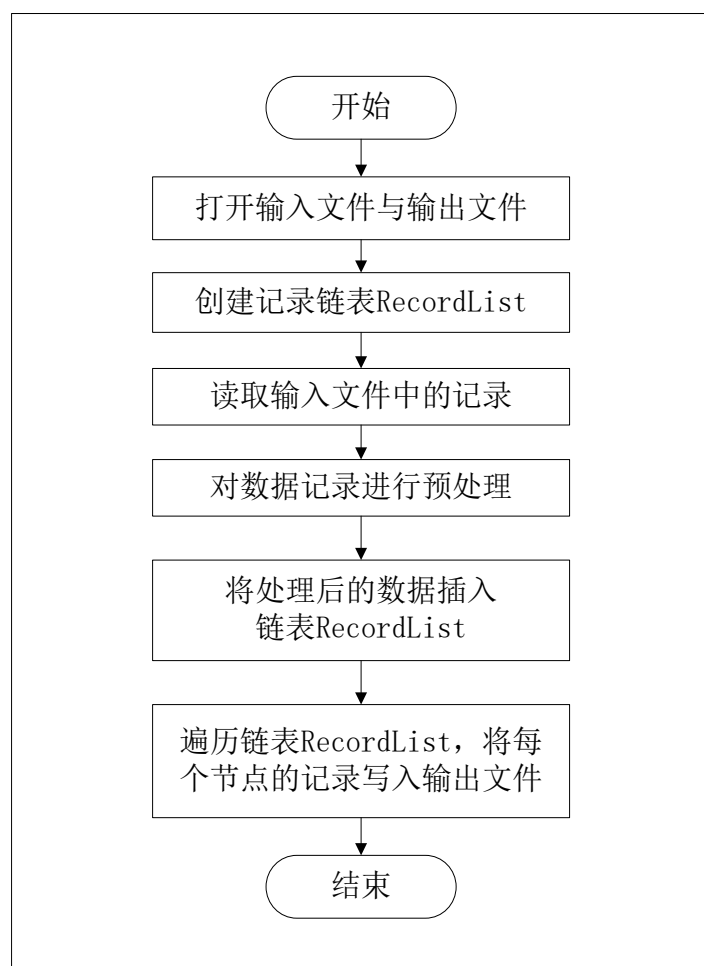


图 2 DataPretreat 程序 main 函数流程图

`main` 函数是 DataPretreat 程序的核心部分。它的流程如图 2 所示。

- (1) 首先打开输入文件和输出文件。输入文件名 `InputFileName` 可以通过 `main` 函数参数 `argv[1]` 获得。输出文件名则在输入文件名后面拼上字符串后缀“_datatreat”即可。
- (2) 使用 STL List 模板创建记录链表 `RecordList`。链表中的每一个节点都是一个

strMyRecord 类型的指针。

- (3) 调用 fgets 函数读取输入文件中的每一条记录。
- (4) 对每一条记录进行数据预处理。预处理工作主要包括以下 3 个方面：
 - a. 删除不需要的属性变量；
 - b. 将符号类型变量数值化；
 - c. 将用百分比表示的变量转换成 0~100 的正整数。
- (5) 调用 STL List 模板的 push_back 函数，将处理完毕的记录插入记录链表 RecordList 的末端。
- (6) 遍历记录链表 RecordList，将每条处理过的记录都写入输出文件。每条记录的不同属性值之间用逗号“,”隔开。

2. Kmeans 程序总体架构

作为本实验的核心部分，Kmeans 程序负责完成两方面的工作：一方面使用 K-Means 算法对训练数据集 kddcup.data_10_percent_datatreat 进行聚类分析，生成聚类树，建立入侵检测模型；另一方面利用入侵检测模型对测试数据集 corrected_datatreat 的数据记录进行类别预测。如表 10-2 所示，Kmeans 程序包含 5 个文件和 3 个类结构。下面将详细介绍各个类的设计与实现方法。

表 10-2 Kmeans 程序组成文件列表

文件名	介绍
Common.h	包含了一些共用的头文件、宏定义。重新定义了结构体 strMyRecord。
Kmeans.h	定义了 CKMeans 类。该类实现了 K-Means 算法，提供递归函数 RunKMeans 作为创建聚类树的方法，提供函数 FindClosestCluster 作为预测数据类别的方法。
Kmeans.cpp	
ClusterTree.h	定义了 ClusterNode 类和 ClusterTree 类。ClusterNode 类描述了聚类树的节点信息，每一个节点代表一个聚类。ClusterTree 类提供了聚类树的操作接口，比如插入操作，匹配最近的节点等等。
ClusterTree.cpp	

3. CKMeans 类的设计与实现

CKMeans 类不仅实现了 K-Means 算法的全部计算过程，而且提供了方便的函数接口。一个 CKMeans 类对象表示聚类树的一个节点。它对输入数据进行聚类分析，将不同类别的数据记录插入不同的子类链表中。如果聚类结果中的某个子类不满足聚类精度的要求，那么就创建一个新的 CKMeans 类对象对该子类继续进行聚类。总之，聚类树的根节点（Root）将整个训练数据集作为输入，利用 K-Means 算法进行一次聚类后，生成若干个子类作为根节点的子节点。不符合聚类精度的子类会继续使用 K-Means 算法对自身链表中的数据进行聚类分析。这样不断地重复下去，直到聚类树中所有叶子节点都符合聚类精度的要求为止。CKMeans 类的定义如下：

```

class CKMeans
{
public:
    //构造函数 1
    CKMeans(ClusterTree* pTree,int KmeansID,int Level,int NumDimensions)
    //构造函数 2
    CKMeans(ClusterNode* pSelf,ClusterTree* pTree,int KmeansID,int Level,
            int NumDimensions,list <strMyRecord*>* pDataList)
    //读取经过数据预处理的记录
    bool ReadTrainingRecords();
    // K-means 算法的第一步：从 n 个数据对象任意选择 k 个对象作为初始聚类中心
    void InitClusters(unsigned int NumClusters);
    // K-means 算法的第二步：把每个对象分配给与之距离最近的聚类
    void DistributeSamples();
    // K-means 算法的第三步：重新计算每个聚类的中心
    bool CalcNewClustCenters();
    //计算指定数据对象与聚类中心的欧几里德距离
    float CalcEucNorm(strMyRecord *pRecord, int id);
    //找到离给定数据对象最近的一个聚类
    int FindClosestCluster(strMyRecord *pRecord);
    //打印聚类中心
    void PrintClusters();
    // K-means 算法的总体过程
    void RunKMeans(int Kvalue);
    //在 KMeans 算法运行之后，查询所有聚类的标签数
    void GetClustersLabel();
    //检查聚类后一类中的分类是否合理
    bool IsClusterOK(int i);
    //获得聚类 i 的链表
    list <strMyRecord*>* GetClusterList(int i);
    //打印本 CKMeans 对象中子类 i 的 label
    void PrintClusterLabel(int i);
    //获得本对象中子类 i 包含不同的 Label 个数
    int GetDiffLabelofCluster(int i);
    //判断是否结束递归过程
    bool IsStopRecursion(int i);
    //创建聚类树节点
    void CreatClusterTreeNode(ClusterNode* pParent);

private:
    //判断该条记录与之前的聚类中心是否完全相同
    bool IsSameAsCluster(int i,strMyRecord* pRecord);
    //为子类 i 的中心赋值
    void EvaluateCluster(int i,strMyRecord* pRecord);

```

```

private:
    FILE *pInFile;                // 输入文件的指针
    list<strMyRecord*> m_RecordsList; // 数据记录链表
    unsigned int m_iNumClusters;    // 聚类的类别数（即 K 值）
    int m_iNumRecords;              // 数据记录的行数
    int m_iNumDimensions;           // 数据记录的维数
    Cluster m_Cluster[MAXCLUSTER]; // 子类数组
    int m_ClusterLevel;             // 聚类对象所处的层次
    int m_KmeansID;                // CKMeans 对象 ID 号
    ClusterTree* pClusterTree;      // 聚类树的指针
    // 本对象孩子节点的指针
    ClusterNode* pClusterNode[MAXLABEL];
    // 与本 CKmeans 对象相关的聚类节点的指针
    ClusterNode* pSelfClusterNode;

};

```

(1) 私有成员变量

文件指针 `pInFile` 用于读取训练数据集文件，并将数据记录保存到链表 `m_RecordsList` 中。`m_iNumClusters` 表示聚类分析中的类别数，即 K-Means 算法中需要预先设定的 K 值大小。`m_iNumRecords` 和 `m_iNumDimensions` 分别表示数据记录的行数与维数（即属性的数目）。子类数组 `m_Cluster` 用于保存聚类结果中每一个子类的信息，比如中心点，成员链表以及成员数目。`m_ClusterLevel` 表示当前 CKMeans 对象在聚类树中所处的层次。`m_KmeansID` 是表示当前 CKMeans 对象的全局唯一标识符。`pClusterTree` 是指向聚类树的指针，而指针 `pSelfClusterNode` 则指向当前 CKMeans 对象在聚类树中对应的节点。指针数组 `pClusterNode` 用于访问 CKMeans 对象的若干子类在聚类树中所对应的节点。

(2) 构造函数

CKMeans 类有两个构造函数。第一个用于生成首个 CKMeans 对象，作为聚类树的根节点。另一个用于生成聚类树中的子节点。如表 10-3 所示，因为根节点与子节点的初始化过程不同，所以在参数设置上也略有差异。根节点没有父节点。它的输入数据来源于训练数据集。普通的子节点需要维护自身与父节点的继承关系。它的输入数据是在父节点的聚类结果中不符合聚类精度的子类。

表 10-3: CKMeans 类构造函数参数对比

参数	说明	构造函数 1	构造函数 2
<code>pSelf</code>	指向聚类树中与自身对象对应的节点	无	有
<code>pTree</code>	指向聚类树	有	有
<code>KmeansID</code>	当前 CKMeans 对象的全局唯一标识符	有	有
<code>Level</code>	当前 CKMeans 对象在聚类树中的层次	有	有
<code>NumDimensions</code>	数据的维数	有	有
<code>pDataList</code>	输入数据链表	无	有

(3) 函数 ReadTrainingRecords

函数 `ReadTrainingRecords` 负责读取 `kddcup.data_10_percent_datatreat` 文件中的数据，将

每一条记录的数据都保存在一个 `strMyRecord` 结构体中，并用链表 `m_RecordsList` 将所有的结构体组织起来。据上所述，只有聚类树的根节点才会调用该函数，而其它节点不需要从训练数据集中读取数据。

(4) 函数 `InitClusters`

函数 `InitClusters` 完成 K-Means 算法的第一步：从 `n` 个数据对象中任意选择 `k` 个对象作为初始聚类中心。函数代码如下：

```
void CKMeans::InitClusters(unsigned int NumClusters)
{
    int i;
    strMyRecord* pRecord;           //遍历记录的指针
    list<strMyRecord*>::iterator RecdListIter; //记录链表的迭代器

    m_iNumClusters = NumClusters;   //对预测的聚类数 k 进行赋值
    RecdListIter=m_RecordsList.begin(); //将 List 迭代器指向链表头部

    //初始化 m_iNumClusters 个类的中心点
    for(i=0; i < m_iNumClusters; i++)
    {
        pRecord = (*RecdListIter);
        //在记录链表中查找出一个与之前的聚类中心数值不同的记录作为中心
        while(IsSameAsCluster(i,pRecord))
        {
            RecdListIter++;
            pRecord = (*RecdListIter);
        }
        //将找到的记录作为一个新的聚类中心保存起来
        //将记录插入该类的成员链表
        m_Cluster[i].MemberList.push_back(pRecord);
        //将该聚类的成员数设置为 1
        m_Cluster[i].iNumMembers = 1;
        //将当前记录的数值赋给第 i 个聚类中心
        EvaluateCluster(i,pRecord);
    }
}
```

函数首先从链表 `m_RecordsList` 的头部开始依次读取每一条记录，选择聚类中心。调用函数 `IsSameAsCluster` 判断当前记录是否与之前选择的聚类中心相同，若是，则放弃当前记录，读取下一条记录；否则，将当前记录作为一个新的聚类中心保存起来。不断重复上面的操作，直到 `k` 个聚类中心选择完毕为止。

(5) 函数 `DistributeSamples`

函数 `DistributeSamples` 完成 K-Means 算法的第二步：把每条数据记录分配给与之距离最近的聚类。具体流程如下：

- a. 清空所有聚类的成员链表。
- b. 迭代器 RecdListIter 指向链表 m_RecordsList 的头部。
- c. 从 m_RecordsList 链表中读取一条记录。
- d. 调用函数 FindClosestCluster，找出与该条记录最近的聚类中心的 ID。
- e. 将该条记录插入到对应聚类中心的成员链表中。
- f. 判断记录是否读取完毕。若是，则退出；否则，迭代器 RecdListIter 指向下一条记录，然后返回第 3 步继续执行。

在第 d 步中，函数 FindClosestCluster 调用了函数 CalcEucNorm 分别计算出当前记录到 k 个聚类中心的距离。然后返回距离最近的聚类 ID。函数 CalcEucNorm 的代码如下：

```
float CKMeans::CalcEucNorm(strMyRecord *pRecord, int id)
{
    double fDist;
    fDist = 0;

    fDist += pow((pRecord->fProtocolType - m_Cluster[id].Center[0]), 2);
    fDist += pow((pRecord->fService - m_Cluster[id].Center[1]), 2);
    fDist += pow((pRecord->fStatusFlag - m_Cluster[id].Center[2]), 2);
    fDist += pow((pRecord->fSrcBytes - m_Cluster[id].Center[3]), 2);
    fDist += pow((pRecord->fDestBytes - m_Cluster[id].Center[4]), 2);
    fDist += pow((pRecord->fFailedLogins - m_Cluster[id].Center[5]), 2);
    fDist += pow((pRecord->fNumofRoot - m_Cluster[id].Center[6]), 2);
    fDist += pow((pRecord->fCount - m_Cluster[id].Center[7]), 2);
    fDist += pow((pRecord->fSrvCount - m_Cluster[id].Center[8]), 2);
    fDist += pow((pRecord->fErrorRate - m_Cluster[id].Center[9]), 2);
    fDist += pow((pRecord->fSameSrvRate - m_Cluster[id].Center[10]), 2);
    fDist += pow((pRecord->fDiffSrvRate - m_Cluster[id].Center[11]), 2);
    fDist += pow((pRecord->fDstHostSrvCount - m_Cluster[id].Center[12]), 2);
    fDist += pow((pRecord->fDstHostSameSrvRate - m_Cluster[id].Center[13]), 2);
    fDist += pow((pRecord->fDstHostDiffSrvRate - m_Cluster[id].Center[14]), 2);
    fDist += pow((pRecord->fDstHostSameSrcPortRate - m_Cluster[id].Center[15]), 2);
    fDist += pow((pRecord->fDstHostSrvDiffHostRate - m_Cluster[id].Center[16]), 2);
    fDist += pow((pRecord->fDstHostSrvErrorRate - m_Cluster[id].Center[17]), 2);

    return float(fDist);
}
```

函数将表 10-1 列出的 18 项属性作为输入，按照前文列出的欧氏距离的计算方法计算出当前记录与指定聚类中心之间的距离。值得注意的是，函数并没有完全按照前文进行计算。因为欧氏距离的计算结果是否开方并不影响数值大小的比较，所以在函数 CalcEucNorm 就省去了开方这一步骤。

(6) 函数 CalcNewClustCenters

函数 CalcNewClustCenters 完成 K-Means 算法的第三步：重新计算每个聚类的中心。函数使用循环分别对 k 个聚类中心重新进行计算。其中每个聚类中心的计算流程如下：

- a. 将临时聚类中心 TempCenter 的每一个属性值都设置为 0。
- b. 遍历当前聚类中心的成员链表，将每一条记录的属性值都加在 TempCenter 对应的属性上。将 TempCenter 中的每一个属性值除以聚类成员数，计算出属性的平均值。
- c. 判断每个平均值与当前聚类中心的属性值是否不同。只要有一个不同，将新计算出的平均值更新为聚类中心，返回 false；否则，聚类中心不变，返回 true。

(7) 函数 RunKMeans

函数 RunKMeans 将前面介绍的三个函数串联起来，实现了 K-Means 算法的整个过程。函数 RunKMeans 的流程如图 3 所示：

- a. 首先调用函数 InitClusters，为本次聚类初始化聚类中心。
- b. 判断 bool 变量 IsFinish 的值是否为 true。若是，则结束本次聚类，跳转至第 5 步；否则继续下面的步骤。
- c. 调用函数 DistributeSamples，将每条记录分配给最近的聚类中心。
- d. 调用函数 CalcNewClustCenters，重新计算聚类中心。如果新的聚类中心与之前的聚类中心相同，则将 bool 变量 IsFinish 设为 true。
- e. 调用函数 GetClustersLabel，打印本次聚类的结果。包括每个聚类的聚类中心，成员数目以及组成情况。
- f. 判断对聚类结果的检查是否结束？利用变量 i 遍历所有的聚类。如果 $i \geq Kvalue$ ，那么所有的聚类检查完毕，跳至第 10 步；否则，继续下面的步骤。
- g. 调用函数 IsClusterOK，判断第 i 个聚类是否符合聚类精度要求？若满足要求，跳至上一步；否则，继续下面的步骤。
- h. 调用函数 IsStopRecursion，判断是否需要对第 i 个聚类继续进行聚类？若是，则继续下面的步骤；否则，跳至第 6 步。
- i. 创建一个新的 CKMeans 对象，对第 i 个聚类的成员继续进行聚类。然后跳转至第 6 步。
- j. 打印相关日志，退出函数。

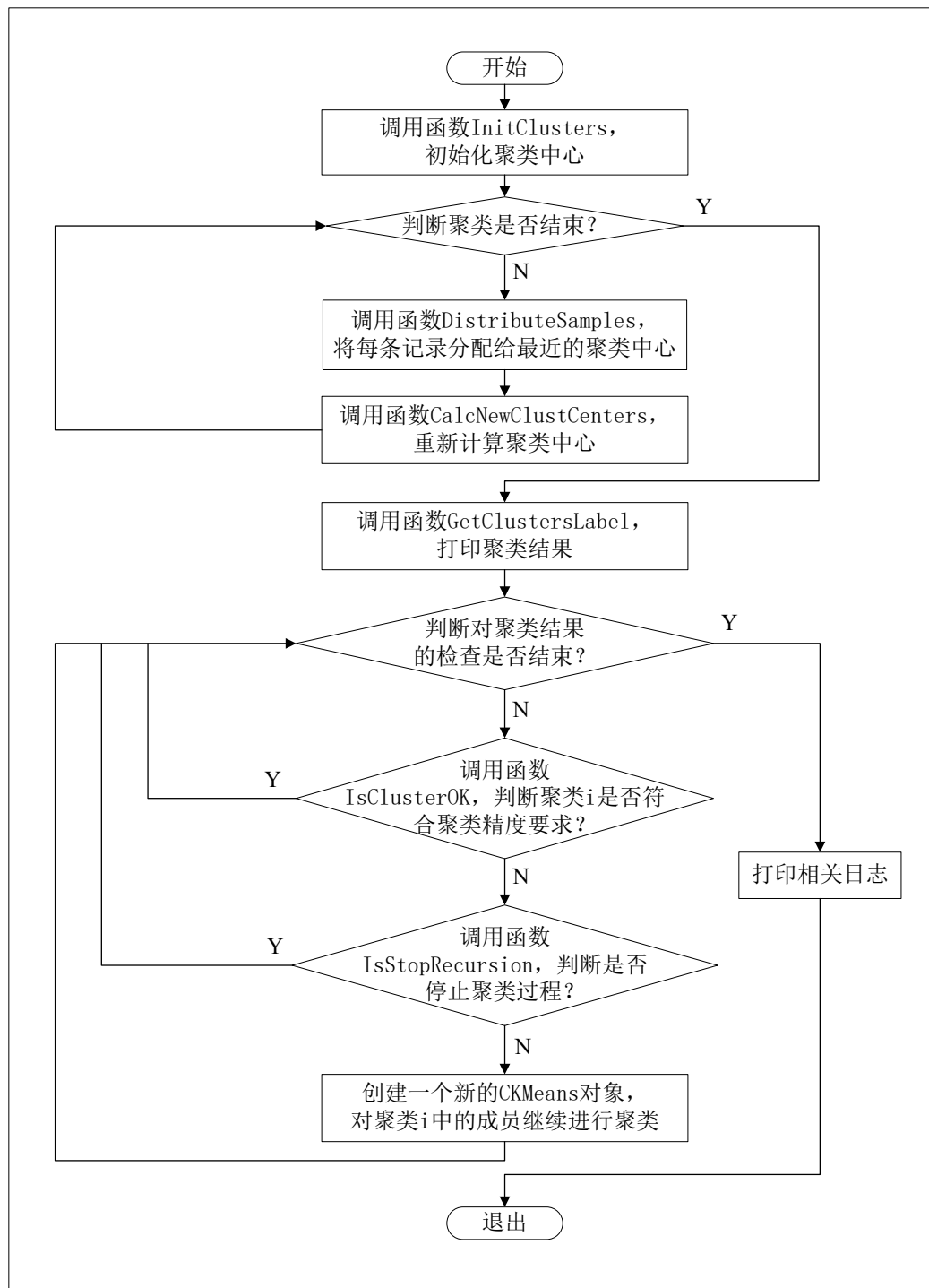


图3 函数 RunKMeans 流程图

4. ClusterNode 类与 ClusterTree 类的设计与实现

(1) ClusterNode 类实现了聚类树中聚类节点的相关操作，为实现 ClusterTree 类提供了底层的操作接口。ClusterNode 类的声明如下：

```

class ClusterNode
{
public:

```



```

//构造函数
ClusterNode() {.....}
//计算该条记录到该节点中心的距离
float CalCenterDistance(strMyRecord* pRecord);
//获得孩子 i 的指针
ClusterNode* GetChildNode(int i);
//获得本节点的聚类标签
int GetClusterNodeLabel();
//递归函数，在以该节点为父亲的子树中，获得与数据记录距离最近的聚类节点
ClusterNode* GetNearestCluster(strMyRecord* pRecord);
//递归函数，打印节点信息
void Print();
//将聚类树输出到日志文件中
void PrintLog();

public:
    float fCenter[MAXDIMENSION]; //聚类节点的中心
    string strPath; //聚类节点的路径
    ClusterNode* pParentNode; //指向这个聚类节点的父节点的指针
    ClusterNode* //指向这个聚类节点的孩子节点的指针
    pChildNode[MAXLABEL];
    float ClusterResult; //聚类精度
    bool IsClusterOK; //聚合结果是否符合标准
    int IsLeaf; //节点类型，是否为叶子节点， 聚类是否正常
    //终止
    //0: 非叶子节点
    //1: 叶子节点且聚类正常结束
    //2: 叶子节点且聚类非正常结束

    int iLabelNum[MAXLABEL]; //记录聚类中各种标签的数目
};

```

(2) ClusterTree 类将所有聚类节点组织在一棵聚类树中，实现了插入聚类节点、匹配距离最近节点等操作。ClusterTree 类的声明如下：

```

class ClusterTree
{
public:
    //构造函数
    ClusterTree()
    {
        pRootNode = new ClusterNode();
        pRootNode->strPath = "0";
    }
    //插入节点
    void InsertNode(ClusterNode* pParent,ClusterNode* pNode,int i);
    //找到与给定记录距离最近的聚类节点

```

```

ClusterNode* FindNearestCluster(strMyRecord* pRecord);
//获得根节点
ClusterNode* GetRootNode();
//打印聚类树
void Print();
//将聚类树输出到日志文件中
void PrintLog();

private:
//根节点指针
ClusterNode* pRootNode;
};

```

聚类树的生成和使用与 Kmeans 程序的整个运行过程是紧密联系在一起的。利用 K-Means 算法对训练数据集进行多次聚类分析的过程也是生成聚类树的过程,而对测试数据集中的记录进行预测的过程其实就是在聚类树中为测试记录寻找最匹配的聚类节点的过程。下面分别对这两个过程进行简单的介绍。

(3) 聚类树的生成

聚类树的生成过程是 Kmeans 程序对训练数据集进行多次聚类分析的过程,也是建立入侵检测模型的过程。它的主要流程如下:

a. 为了对训练数据集进行聚类分析,需要在 Kmeans 程序中创建了一个 CKMeans 对象。伴随着这一操作,聚类树完成根节点的插入工作并且将根节点的路径设置为“0”。这些工作都是在 ClusterTree 类的构造函数中实现的。

b. CKMeans 对象调用 RunKMeans 函数对训练数据集进行分析。当聚类完成以后,会调用函数 CreatClusterTreeNode 为结果中的每一个聚类创建节点,并将它们作为自己的孩子节点插入聚类树中。例如,在对训练数据集进行第一次聚类后,生成了 5 个聚类。那么就在根节点下插入 5 个子节点,并将这 5 个子节点的路径分别设置为“0.0”、“0.1”、...

c. 调用函数 IsClusterOK 进一步判断每一个聚类是否符合聚类精度要求。如果符合要求,那么不需要再进行任何操作;否则,创建一个新的 CKMeans 对象,将该聚类的所有成员作为输入数据,开始新的聚类过程。

由此看出, Kmeans 程序在运行过程中会重复地执行第 2 步和第 3 步,不断地向聚类树中插入新的节点。同时也不难看出,在聚类树中,子节点的路径总是在父节点路径的基础上加上新的编号。

(4) 匹配距离最近的聚类节点

在聚类树中为一条数据记录寻找距离最近的聚类节点的过程实际上就是为测试数据预测类别的过程。在编程中采用递归的方法,从聚类树的根节点开始,不断地向下寻找距离最近的节点。ClusterTree 类提供了函数 FindNearestCluster 作为对外接口,而 ClusterNode 类的 GetNearestCluster 函数则实现了整个递归过程。上述两个函数的定义如下。

```

/*****接口函数, 找到与给定记录距离最近的聚类 节点*****/
ClusterNode* ClusterTree::FindNearestCluster(strMyRecord* pRecord)

```

```

{
    ClusterNode* pNearestNode;
    //调用了根节点的 GetNearestCluster 函数
    pNearestNode = pRootNode->GetNearestCluster(pRecord);
    return pNearestNode;
}

/** 递归函数，在以该节点为根的子树中，获得与数据记录最近的聚类节点 */
ClusterNode* ClusterNode::GetNearestCluster(strMyRecord* pRecord)
{
    int i;
    float MinDistance, TmpDistance;           //最短距离，临时距离
    ClusterNode* pNearestNode;                //距离最近节点指针
    ClusterNode* pTmpNode;                    //临时节点指针

    //判断是否为叶子节点
    if(IsLeaf > 0)
    {
        pNearestNode = this;
    }
    else
    {
        pNearestNode = this;
        //计算本节点的聚类中心与记录的距离 d1
        MinDistance = CalCenterDistance(pRecord);
        //判断是否有孩子节点，如果有，调用递归函数，获得孩子节点所在的子树中
        //离数据距离最近的节点指针
        for(i=0; i<MAXLABEL; i++)
        {
            if(pChildNode[i] != NULL)
            {
                pTmpNode = pChildNode[i]->GetNearestCluster(pRecord);
            }
            //计算最近子节点与数据的距离 d2
            TmpDistance = pTmpNode->CalCenterDistance(pRecord);
            //如果 d2 < d1，返回本节点的指针，否则，返回子节点指针
            if(TmpDistance < MinDistance)
            {
                pNearestNode = pTmpNode;
                MinDistance = TmpDistance;
            }
        }
    }
    //返回最近节点的指针
    return pNearestNode;
}

```

}

在 `FindNearestCluster` 函数中，仅仅调用了根节点的 `GetNearestCluster` 函数，在整棵聚类树中寻找与记录距离最近的节点。而函数 `GetNearestCluster` 通过自身不断地递归调用完成了对整棵聚类树的查询工作，并返回距离最近节点的指针。当某个节点调用 `GetNearestCluster` 函数时，它只负责在以自身为父节点的子树中向下寻找距离最近的节点。因此，如果一个父节点需要向下寻找距离最近的节点，首先要让所有孩子节点分别在自身的子树中找出距离最近的节点，然后将这些点的距离与父节点的距离进行比较，从而找出距离最近的节点。函数 `GetNearestCluster` 就严格地遵循了上述递归思想。如图 4 所示，函数 `GetNearestCluster` 的执行过程分为以下 8 个步骤。

- a. 判断当前节点是否为叶子节点。若是，将距离最近节点的指针 `pNearestNode` 指向当前节点并返回；否则，继续下面的步骤。
- b. 计算出当前节点与数据记录之间的距离 `MinDistance`。
- c. 判断当前节点是否有孩子节点？若有，继续下面的步骤；否则跳至第 8 步。
- d. 调用 `GetNearestCluster` 函数在当前孩子节点的子树中找出距离最近的节点，并返回指针 `pTmpNode`。
- e. 调用 `CalCenterDistance` 函数计算 `pTmpNode` 所指向的节点与数据记录之间的距离 `TmpDistance`
- f. 判断距离 `TmpDistance` 是否小于距离 `MinDistance`？若是，继续下面的步骤；否则，跳至第 3 步。
- g. 将指针 `pTmpNode` 所指向的节点更新为距离最近的节点。用指针 `pNearestNode` 指向该节点，并用 `MinDistance` 表示最近距离的大小。然后跳至第 3 步。
- h. 返回最近节点的指针 `pNearestNode`，退出函数。

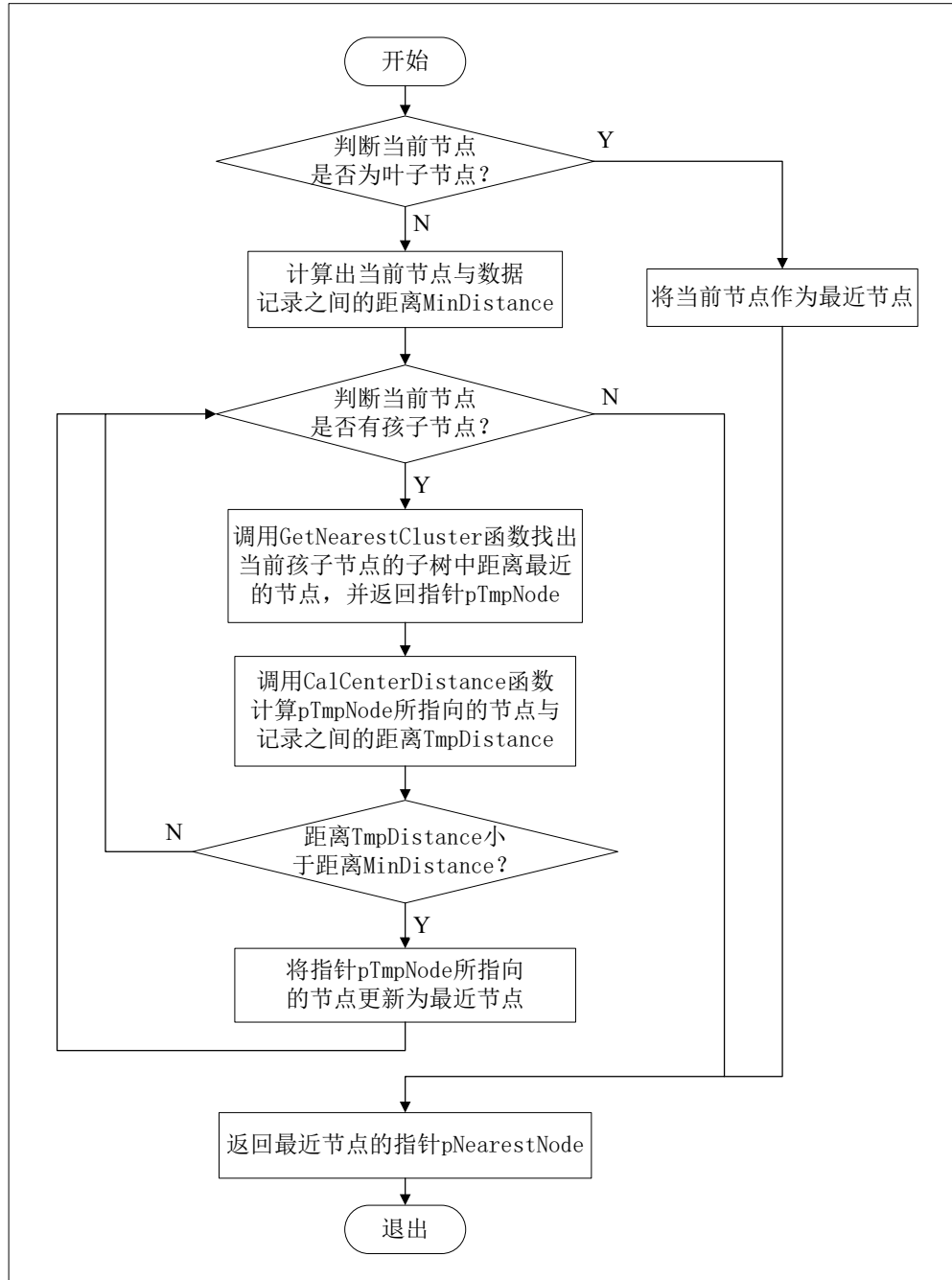


图4 函数 GetNearestCluster 流程图

5. Kmeans 程序 main 函数的设计与实现

在 Kmeans 程序中，main 函数通过调用 ClusterNode 类与 ClusterTree 类所提供的接口函数将两者有机地结合起来。它不仅使用 K-Means 算法对测试数据集进行多次聚类分析，建立了入侵检测模型，还利用该模型对测试数据集中的数据进行类别预测。如图 5 所示，main 函数的执行过程分为以下 8 个步骤。

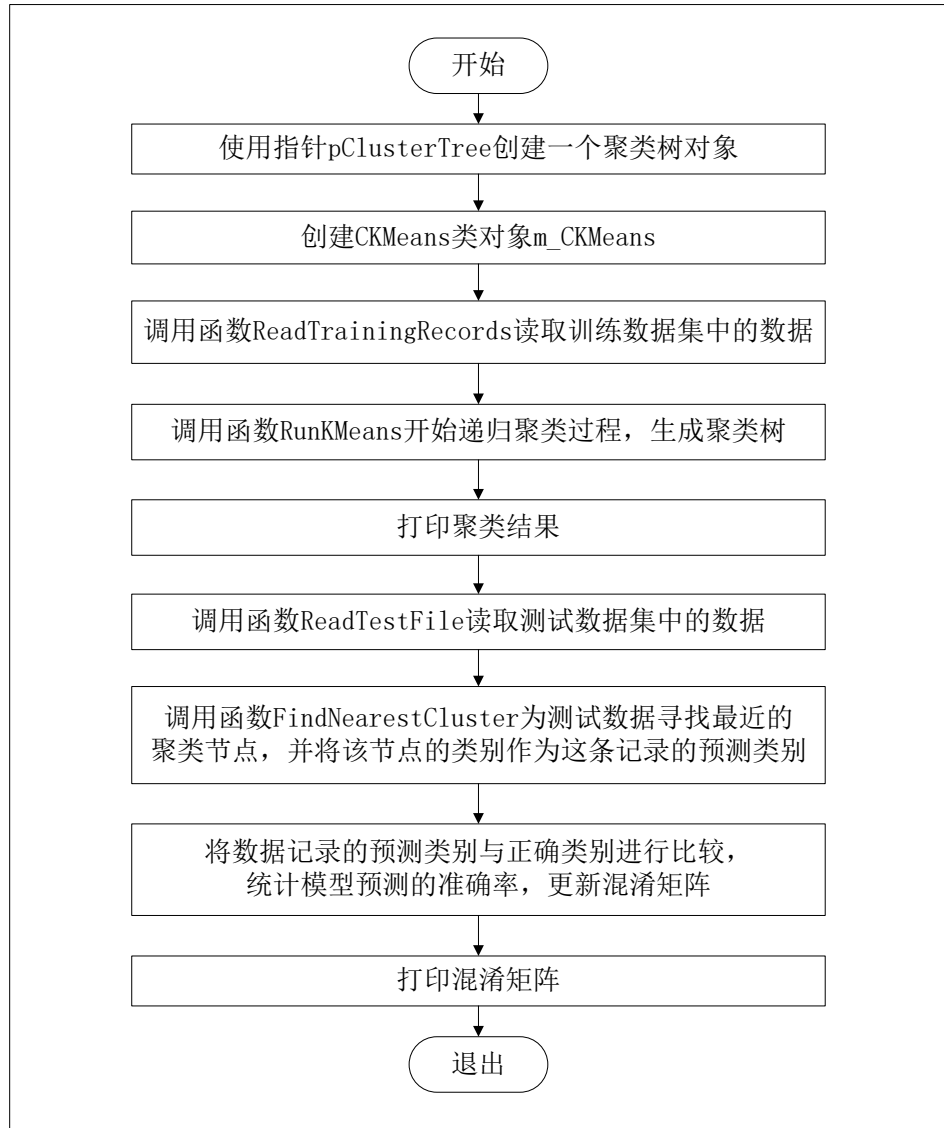


图 5 Kmeans 程序 main 函数流程图

- (1) 分别创建 CKMeans 类与 ClusterTree 类的对象。用指针 pClusterTree 指向聚类树。将 CKMeans 类对象 m_CKMeans 的 KmeansID 设置成 1。
- (2) 调用 m_CKMeans 对象的 ReadTrainingRecords 函数，从数据预处理后的训练数据集文件 kddcup.data_10_percent_datatrea 中读取数据。
- (3) 将 K 值设置成 5，调用 m_CKMeans 对象的 RunKMeans 函数开始递归聚类过程。在此过程中会产生新的聚类节点，需要为这些节点创建新的 CKMeans 类对象，同时将这些节点插入聚类树中。
- (4) 整个聚类过程结束后，将聚类信息输入到日志文件 Log.txt 中。
- (5) 调用函数 ReadTestFile 将测试数据集文件 corrected_datatrea 中的测试数据读入链表 pTestRcdList 中。
- (6) 遍历链表 pTestRcdList。调用函数 FindNearestCluster 为每一条记录寻找聚类树中距离最近的聚类节点。将该聚类节点的类别作为记录的预测类别。

(7) 将测试数据的预测类别与真实类别进行比较，将结果记录到 Result.txt 文件中。统计预测结果的准确性，并更新混淆矩阵。

(8) 在所有的测试数据预测完毕后，打印混淆矩阵。

1.4 扩展与提高

1. 聚类精度的选取对入侵检测模型的影响

根据 K-Means 算法扩展这一小节中的介绍，聚类精度 (Cluster Precision) 是指一个聚类中干扰数据占主类别数据的比例。聚类精度的大小直接决定了一个聚类是否需要再次使用 K-Means 算法进行聚类分析。但是如何设定聚类精度一直是影响入侵检测模型准确性的重要因素。聚类精度设置过高，会造成类别划分不准确的问题。一个聚类中往往会包含多种类别的数据。这些干扰数据越多，入侵检测模型的准确性就越低。因此，为了提高模型的准确性，通常聚类精度都会选择较小的数值。但是如果聚类精度降低，Kmeans 程序的递归次数将会增加，所占用的资源也会随之增加。有时候，一味的追求较小的聚类精度，可能会造成整个程序无法停止，聚类树的生成过程无法收敛的情况。综上所述，合理地选择聚类精度的标准有助于我们高效准确的建立入侵检测模型。

然而，只考虑聚类精度是不够的。在聚类精度符合要求的情况下，还会出现一些特殊的问题。例如，假设两个聚类的聚类精度都是 0.1，一个聚类包含了 10 条干扰数据和 100 条主类别数据，而另一个聚类包含了 1000 条干扰数据和 10000 条主类别数据。显然，前者不需要再继续聚类，而后者虽然达到了聚类精度的标准，但仍包含了 1000 条干扰数据。对于这个聚类来说，拥有如此多的干扰数据，完全可以再进行一次聚类分析将这些干扰数据分离开来。鉴于上述考虑，规定聚类中干扰数据的数量可以弥补只根据聚类精度来判定程序是否继续递归的不足，可以有效地防止特殊情况的发生。

根据上述两点，本书在编程过程中将根据聚类节点在聚类树中所处的不同层次设置对聚类精度与干扰数据的数目设定不同的标准。具体方案如下：

(1) 首先在每一个 CKmeans 对象中用 m_ClusterLevel 来表示该对象在聚类树中所处的层次。规定，根节点的层次为 1，各级子节点的层次依次递增。

(2) CKmeans 类的 IsClusterOK 函数会计算当前的聚类精度进行并将结果保存在变量 Result 中。然后根据 m_ClusterLevel 变量的不同数值对 Result 进行检查，做出相应的操作。根据聚类层次是否大于 INTERLEVEL (默认设置为 3)，选择聚类精度时分别按照以下 2 种标准。

a. 当 m_ClusterLevel 小于等于 INTERLEVEL 时，首先判断干扰数据的数目是否大于 100，若是，返回 false，要求继续聚类；否则，继续判断聚类精度是否小于 CLUSTER_PRECISION (程序中设为 0.1)，若是，返回 true，不再进行聚类；否则，返回 false。

b. 当 m_ClusterLevel 大于 INTERLEVEL 时，首先判断干扰数据的数目是否大于 500，若是，返回 false，要求继续聚类；否则，继续判断聚类精度是否符合要求。利用下式来判断聚类精度是否小于设定的标准，若是，则返回 true，不再进行聚类；否则返回 false。

$$\text{Result} < (\text{m_ClusterLevel} - \text{INTERLEVEL}) \times \text{CLUSTER_PRECISION}$$

上述方案对聚类精度的设置如图 6 所示。图中 CP 表示当前聚类层次所要求的聚类精度。可以看出，随着聚类层次的不增大，聚类精度的要求也越来越低。这样既保证了在程序刚开始运行时生成的聚类有较低的精度，也保证了随着聚类层次地不断增加整个聚类过程逐渐达到收敛的状态。

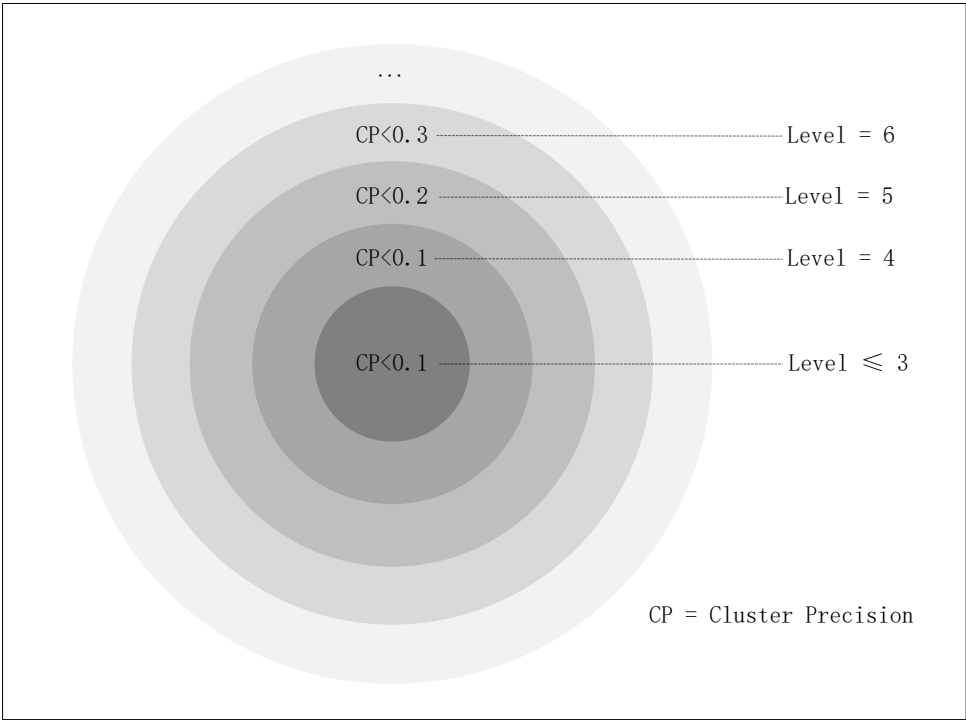


图 6 聚类精度设置图

2. Linux 平台下的入侵检测工具

目前，Linux 平台下的入侵检测工具有很多，其中具有代表性的有 Snort 和 PSAD。这两款软件具有各自不同的特点，下面对它们进行简单的介绍。

1. Snort

早在 1998 年，Martin Roesch 编写了一个开放源代码的入侵检测工具，并命名为 Snort。迄今为止，Snort 已发展成为一个具有跨多平台，提供实时流量分析，记录网络 IP 数据报等诸多特性的强大的入侵检测与防御系统。

Snort 有三种工作模式：嗅探器、数据包记录器、网络入侵检测系统。嗅探器模式仅仅是从网络上读取数据包并作为连续不断的流显示在终端上。数据包记录器模式把数据包记录到硬盘上。网路入侵检测模式是最复杂的，而且是可配置的。用户可以让 snort 分析网络数据流以匹配用户定义的一些规则，并根据检测结果采取一定的动作。

网络入侵检测系统（NIDS）是 Snort 最重要的工作模式。可以使用下面命令行来启动这一模式。

```
./snort -d -h 192.168.1.0/24 -l ./log -c snort.conf
```


其中 `snort.conf` 是规则集文件。`Snort` 会对每个包和规则集进行匹配，发现这样的包就采取相应的行动。上述命令是使用 `Snort` 作为网络入侵检测系统最基本的形式，日志符合规则的包，以 ASCII 形式保存在有层次的目录结构中。

`Snort` 采用规则匹配的方式检测入侵数据包，因此 `Snort` 规则的编写十分重要。`Snort` 使用一种简单的，轻量级的规则描述语言来编写规则。在使用这种语言时，需要掌握以下原则。

`Snort` 规则分为两个逻辑部分：规则头和规则选项。规则头包含规则的动作，协议，源和目标 IP 地址与网络掩码，以及源和目标端口信息；规则选项部分包含报警消息内容和要检查的包的具体部分。下面是一个规则范例：

```
alert tcp any any -> 192.168.1.0/24 111 (content:"|00 01 86 a5|"; msg: "mountd access");
```

括号前的部分是规则头(Rule Header)，包含的括号内的部分是规则选项(Rule Options)。规则选项部分中冒号前的单词称为选项关键字 (Option Keywords)。注意，不是所有规则都必须包含规则选项部分，选项部分只是为了对符合规则的数据包所进行的操作进行更加严格的定义，比如收集、报警、丢弃等等。组成一个规则的所有元素对于指定的操作都必须是真的。当多个元素放在一起时，可以认为它们之间是逻辑与 (AND) 的关系。同时，`Snort` 规则库文件中的不同规则之间可以认为是逻辑或 (OR) 的关系。

2. PSAD

PSAD 是端口扫描攻击探测器 (The Port Scan Attack Detector) 的简称。作为一款优秀的入侵检测工具，它能够检测出不同类型的可疑流量，比如 Nmap 的端口扫描，DDos 攻击以及对系统某个协议的暴力破解等。通过分析防火墙的日志，PSAD 不仅能够检测出某种可疑的攻击，而且还能够通过改变防火墙的规则来相应这一攻击。

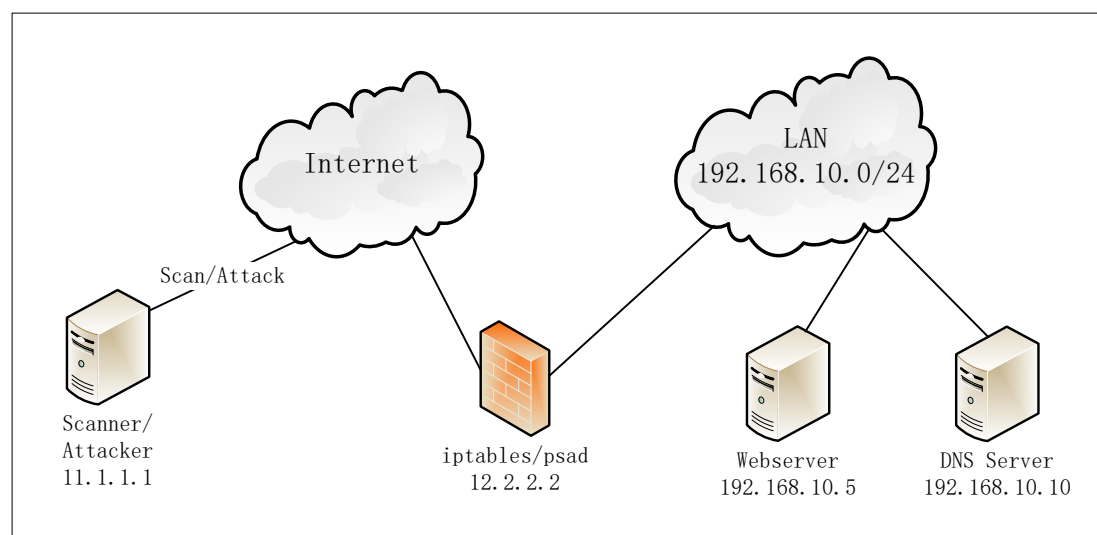


图 7 PSAD 部署图

PSAD 与 `iptables` 和 `Snort` 联系非常紧密。PSAD 在 Linux 系统中运行了 3 个轻量级的系统守护进程。这些守护进程可疑分析 `iptables` 的日志信息并检测出端口扫描以及其它可疑的流量。图 7 是 PSAD 部署在防火墙 `iptables` 上的典型结构图。从图中可以看出 PSAD 能够快速访问防火墙的日志数据。PSAD 还可以利用 `Snort` 来检测各种后门程序，DDos 工具，以及高级的端口扫描器的探测行为。通过与 `Snort` 结合，PSAD 能够检测出 `Snort` 规则集中

所描述的各种攻击。此外，PSAD 还能够利用 TCP SYN 数据包的报头字段取得发起扫描行为的远程主机的指纹。

另外，PSAD 还提供了数据输出的接口。用户可以将 PSAD 中的数据导入到软件 AfterGlow 和 Gnuplotd 中。通过这些软件绘制的各种图表可以进一步分析出到底是谁正在攻击防火墙。